

INTERNATIONAL UNIVERSITY

IT096IIU – NETCENTRIC

LAB 01 – GO FOUNDATION

FOOD ORDERING SYSTEM

Phạm Gia Thịnh - ITITI24061
Huỳnh Ngọc Thủy Tiên - ITITI24065

1 THINK

1. What information belongs to a `FoodItem`?

- 1) `FoodItemID` string
- 2) `FoodName` string
- 3) `Price` float64
- 4) `Category` string
- 5) `Available` bool

2. What information belongs to an `Order`?

- 1) `OrderID` string
- 2) `Items` `map[string]int` (to contain the food names and their quantities)

3. Can an order contain several different food items?

Yes, an order can contain multiple different food items, which is why we store them in a data structure like a map to calculate a combined subtotal.

4. How should quantity be represented?

As an integer (`int`), representing the multiplier for the item's price when added to the order.

5. Where should food items be stored?

They should be stored in memory using Go data structures (like a slice for the master menu and a map for quick lookups).

6. Where could a map be useful?

A map, for example `map[string]FoodItem`, is great for searching for a food item by `FoodName` quickly without looping through a whole list.

7. What should happen when requested food does not exist?

The system will reject the request and notify with the screen: item not found for the user and have a redirect button to the main page or the catalog page show "No item found" and the user can continue to search for another item.

8. What should happen when quantity is 0 or negative?

When the quantity is 0, we can pull down the item, notify unavailable or sold out for the user. When the quantity is negative, it must reject this invalid quantity in the system.

9. What should happen when a food item is unavailable?

The system rejects any request looking for this food item.

10. Which functions/methods should be responsible for business rules?

Functions like quantity checking for an item (`isValidQuantity`) or total cost of order (`printOrderSummary`).

Create a short design containing:

- **FoodItem**

```
type FoodItem struct {  
    FoodItemID string  
    FoodName string  
    Price float64  
    Category string  
    Available bool  
}
```
- **Order**

```
type Order struct {  
    OrderID string  
    Items map[string]int  
}
```
- **at least one slice and one map**
Slice: `[]FoodItem` to store the master menu. This makes it very easy to display all available food items by looping through them.
Map: `map[string]FoodItem` to act as a quick lookup index. When a user types a name to add to their order, we instantly check if it exists.
- **important functions/methods**
Since we need to display, search, add, show order summary:
`listInventory(), searchFood(foodName string), addItemToOrder(), printOrderSummary().`
- **two possible designs**
Design A: The Order uses two parallel slices (one for items, one for quantities).
Design B: The Order uses a map where the key is the food name, and the value is the quantity (`map[string]int`).
- **the chosen design and one reason**
Design B is better because if a customer orders a food twice, we can easily update the existing integer quantity in the map rather than adding a second food entry to a slice.
- **four predicted failure cases**
 - 1) User enters a negative quantity.
 - 2) User enters a quantity of 0.
 - 3) User orders an item that exists but is marked **Available: false**.
 - 4) User orders an item that does not exist in the system at all.

2 BUILD

We implemented the system using Go. We utilized a slice (`menuSlice []FoodItem`) to act as the primary master list for displaying the menu. We also mapped this slice into a map (`menuMap map[string]FoodItem`) to provide instant $O(1)$ lookups when searching for food by name or validating an order.

Our `Order` struct tracks selected items using a `map[string]int`, mapping the food's exact name to the requested quantity. Validation rules are handled strictly inside the `addItemToOrder` function.

3 VERIFY

Case	Expected behavior	Result
Normal order	Correct subtotal	Passed (Total calculated correctly)
Quantity = 1	Accepted	Passed
Quantity = 0	Rejected	Passed (Caught by isValidQuantity)
Negative quantity	Rejected	Passed (Caught by isValidQuantity)
Unavailable food	Rejected	Passed (Checked Available boolean)
Food not found	Handled safely	Passed (Returns error safely)
Several different items	Correct total	Passed (Iterates map correctly)

Intentional Defect

Defect introduced: Inside the `printOrderSummary` function, we changed the subtotal calculation from `lineTotal := food.Price * float64(qty)` to `lineTotal := food.Price + float64(qty)`.

Explanation & Fix: This defect caused the system to add the quantity to the price instead of multiplying it (e.g., $50000 + 2$ instead of $50000 * 2$). We identified the incorrect total in the console, fixed the math operator back to `*`, and retested successfully.

4 ADAPT

Requirement: Orders with a subtotal of at least 300,000 VND receive a 10% discount.

Reflection: The original design was very easy to change because the subtotal calculation was isolated inside the `printOrderSummary` function. We simply added a condition to check if `subtotal >= 300000` before printing. However, if several complex discount rules were introduced (e.g., item-specific discounts or VIP tiers), this function would become bloated and difficult to maintain. We would need to extract the discount logic into a dedicated `CalculateDiscount()` interface or function.

5 AI Collaboration Log

#	What we asked AI	What AI suggested	What we changed/verified
1	Asked AI to review our initial code against the lab requirements.	Pointed out we used <code>int</code> for Price instead of <code>float64</code> , and searched by ID instead of Name.	We changed Price to <code>float64</code> and updated our maps to use the Food Name as the key.
2	Asked for a function to calculate the total and print an order summary.	Generated a <code>printOrderSummary</code> function that iterates through the order, calculates line subtotals by converting quantity to <code>float64</code> , and calculates the grand total.	After reviewing the generated code, we verified the logic met our expectations. It correctly pulls data from the inventory and calculates the receipt by multiplying the item price with the ordered quantity.
3	Asked for a function to display the available menu.	Suggested a <code>listInventory</code> function that iterates through the inventory map, and a secondary <code>getInventoryList</code> function to return a slice.	We adapted the <code>listInventory</code> logic to iterate over our primary slice to ensure consistent menu ordering. Additionally, we discarded the suggested <code>getInventoryList</code> function, as it offered no practical utility for our current system design.